Multitasking in Java

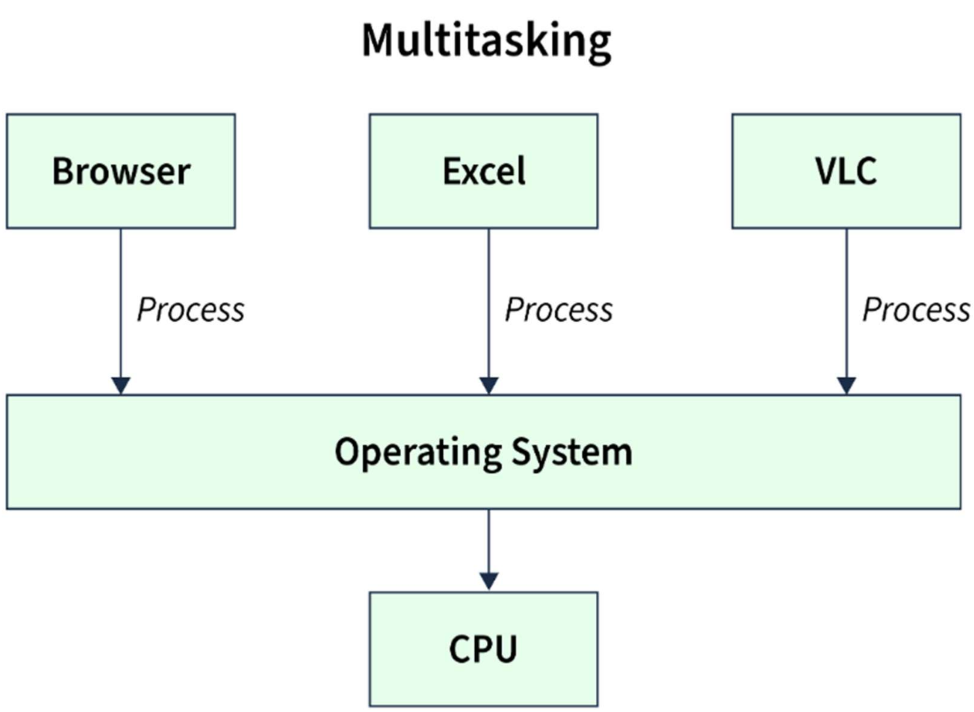
What is Multitasking :-

- Multitasking in Java means **executing multiple tasks at the same time** to make efficient use of the CPU.

Multitasking

Multiple + **Task**

- **Example of Multitasking :-**



Types of Multitasking :-

➢ The execution of multiple tasks simultaneously, including process-based and thread-based tasks.

Multitasking

Multiprocessing (Process Based Multitasking)

Multithreading (Thread Based Multitasking)

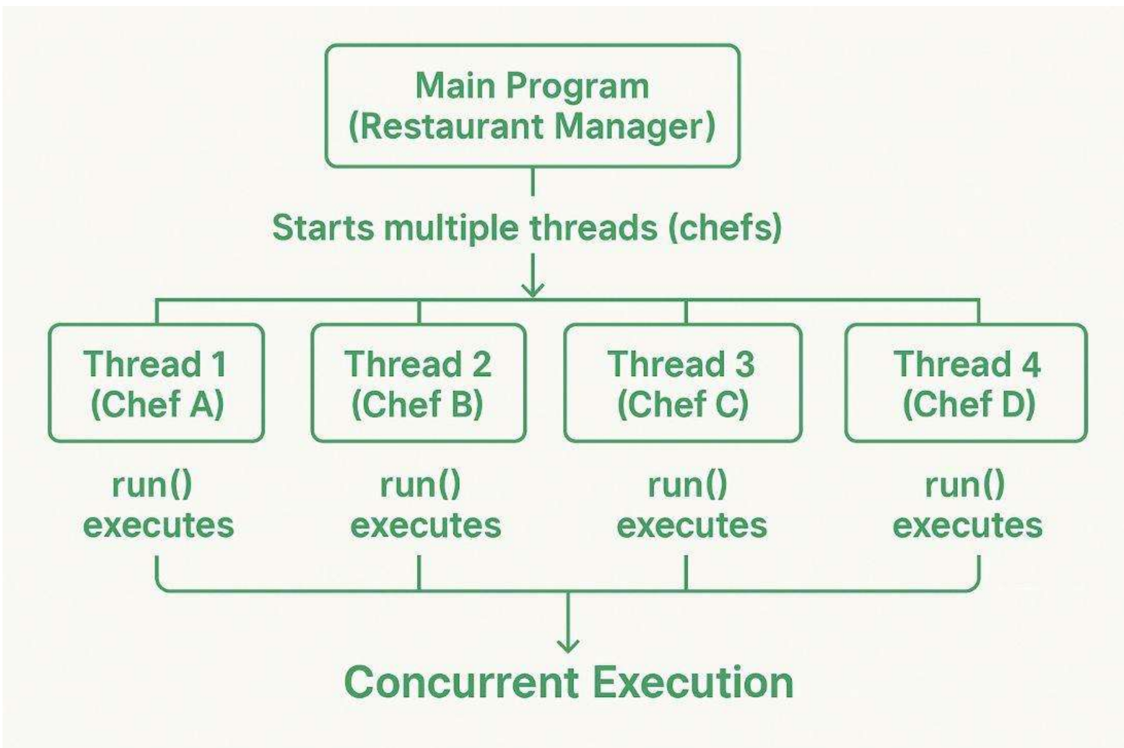
What is Multiprocessing :-

- Multiprocessing means **executing two or more processes simultaneously** to increase the performance of a system.
- Each process has its **own memory space** and **runs independently**.
- In multiprocessing each process has **separate memory**.
- There are using **multiple CPUs or cores** for true parallel execution.
- Example : Running a **Java program**, a **browser**, and a **video player** simultaneously.

What is Multithreading :-

- Multithreading in Java is a feature that allows **multiple parts of a program (threads)** .
- To **run concurrently improving performance** and **CPU utilization.**

Example of Multithreading :-



Why Use Multithreading :-

- To perform **multiple tasks at once**
- To make applications **faster and more responsive**
- To utilize **CPU cores** efficiently
- To perform **background tasks** (like **saving files**, **downloading**, etc.)

Methods in Multithreading :-

Method	Purpose
getName	Obtain a thread's name
getPriority	Obtain a thread's priority
isAlive	Determine if a thread is still running
join	Wait for a thread to terminate
run	Entry point of a thread
sleep	Suspend a thread for a period of time
start	Start a thread by calling its run method

Difference between Multiprocessing vs Multithreading :-

Basis	Multiprocessing	Multithreading
Definition	Multiple processes run at once	Multiple threads run inside a single process
Memory	Each process has its own memory	Threads share same memory
Communication	Harder (uses inter-process communication)	Easier (threads share variables)
Speed	Slower (more overhead)	Faster and lightweight
Crash Effect	One process crash doesn't affect others	One thread crash can affect the process

Implementation of Multithreading :-

Input :-

```java
package Multi_Threading;

public class threadinf_methods {

    public static void main(String[] args) {

        Thread t1=new Thread();

        t1.setName("audio_thread");
        System.out.println("Name : "+t1.getName());

        t1.setPriority(3);
        System.out.println("Priority : "+t1.getPriority());

        System.out.println("Id : "+t1.getId());
        System.out.println("Min : "+Thread.MIN_PRIORITY);
        System.out.println("Norm : "+Thread.NORM_PRIORITY);
        System.out.println("Max : "+Thread.MAX_PRIORITY);

        System.out.println(t1.isAlive());
        System.out.println(t1.isVirtual());
        System.out.println(t1.isDaemon());

    }
}
```

Output :-

```
Name : audio_thread
Priority : 3
Id : 21
Min : 1
Norm : 5
Max : 10
false
false
false
```

Input :-

```
package Multi_Threading;

//1.Extends the thread class
public class multithreading_1 extends Thread
{
    //2.Override run method or Write down the run method for perticular subtask
    public void run()
    {
        System.out.println("Thread 1 is running");
    }

    public static void main(String[] args) {
        //3.Create the object of class
        multithreading_1 t1=new multithreading_1();
        //4.Execute the thread
        t1.start();
    }
}
```

Output :-

```
Thread 1 is running
```